

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Regional Boyacá · Centro Industrial de Mantenimiento y Manufactura – CIMM
Tecnólogo en Análisis y Desarrollo de Software – ADSO



TALLER PRÁCTICO

Pruebas de Software con JUnit 5

Pruebas Unitarias e Integración

Aplicado al Sistema de Gestión de Vacunación

Proyecto base:	Sistema de Gestión de Vacunación
Paquete base:	sena.adso.captcha
Framework de pruebas:	JUnit 5 (junit-jupiter 5.10.2)
BD para integración:	H2 en memoria (no requiere MySQL en pruebas)
Tipo de pruebas:	Unitarias (Unit) + Integración (Integration)
Duración estimada:	12 horas — 3 sesiones de laboratorio
Versión:	1.0 – 2026

1. Introducción — Probar el Sistema de Vacunación

Ya tienes el sistema de vacunación funcionando. El siguiente paso profesional es garantizar que sigue funcionando correctamente cuando se modifique. Las pruebas automatizadas con JUnit 5 permiten verificar cada clase del sistema de forma rápida y repetible, sin necesidad de abrir el navegador y probar manualmente cada módulo.

¿Qué vamos a probar del sistema de vacunación?

Las pruebas se organizan en dos tipos:

PRUEBAS UNITARIAS — prueban una sola clase de forma aislada, sin BD ni red.

- Paciente.java → getNombreCompleto(), validaciones de documento
- Vacuna.java → getInfoCompleta(), estaVencida(), validaciones de lote
- Usuario.java → getNombreCompleto(), esMedico(), esEnfermero()
- ValidadorDocumento.java → clase nueva que crearemos para validar cédulas

PRUEBAS DE INTEGRACIÓN — prueban el DAO conectado a una BD H2 real.

- PacienteDAO.java → insertar, listar, buscar, eliminar
- VacunaDAO.java → insertar, buscar por lote, UNIQUE constraint
- RegistroVacunacionDAO.java → insertar registro, listar por paciente

2. Conceptos Clave de JUnit 5

2.1 Anotaciones que usaremos

Anotación	Cuándo se ejecuta	Ejemplo en vacunación
@Test	Al correr las pruebas	@Test void getNombreCompleto_debeUnirNombresYApellidos()
@BeforeEach	Antes de CADA prueba	Crear un Paciente nuevo para cada test
@AfterEach	Después de CADA prueba	Cerrar la conexión H2 en pruebas de integración
@BeforeAll	Una sola vez antes de TODAS	Crear la tabla pacientes en H2
@AfterAll	Una sola vez después de TODAS	Eliminar la tabla y cerrar H2

Anotación	Cuándo se ejecuta	Ejemplo en vacunación
@DisplayName	—	@DisplayName("Vacuna vencida debe retornar true en estaVencida()")
@Nested	—	Agrupar tests de insertar, actualizar y eliminar de PacienteDAO
@ParameterizedTest	Varias veces con distintos datos	Probar documentos válidos e inválidos
@ValueSource	—	@ValueSource(strings={"1052345678","80456789","CC-123"})
@CsvSource	—	Probar múltiples vacunas con distintos datos de entrada

2.2 Assertions más usados

Assertion	Qué verifica — ejemplo con vacunación
assertEquals(esperado, actual)	assertEquals("Carlos Pedraza", paciente.getNombreCompleto())
assertNotNull(objeto)	assertNotNull(vacunaDAO.buscarPorLote("PF001"))
assertNull(objeto)	assertNull(pacienteDAO.buscarPorDocumento("00000000"))
assertTrue(condicion)	assertTrue(vacuna.estaVencida())
assertFalse(condicion)	assertFalse(usuario.esEnfermero()) — cuando el rol es MEDICO
assertThrows(Excepcion.class, ...)	assertThrows(IllegalArgumentException.class, () -> new Paciente(null,...))
assertAll(ejecutables...)	Verificar todos los campos de un Paciente a la vez
assertIterableEquals(lista1, lista2)	Verificar que listarTodos() retorna los registros insertados

2.3 Patrón AAA — Arrange, Act, Assert

Todas las pruebas bien escritas siguen este patrón de tres pasos. Separar los bloques con comentarios mejora la legibilidad y facilita el mantenimiento:

```
// Ejemplo real con la clase Vacuna del sistema
@Test
@DisplayName("getInfoCompleta() debe retornar nombre, lote y laboratorio")
void getInfoCompleta_debeRetornarDatosFormateados() {

    // ARRANGE — preparar el objeto de prueba
    Vacuna vacuna = new Vacuna(1, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-31");

    // ACT — ejecutar el método que queremos probar
```

```
String resultado = vacuna.getInfoCompleta();

// ASSERT - verificar el resultado esperado
assertEquals("COVID-19 Pfizer (Lote: PF001, Lab: Pfizer)", resultado);
}
```

3. Configuración del Proyecto para Pruebas

3.1 Agregar dependencias en pom.xml

El pom.xml del sistema de vacunación ya tiene el driver MySQL y la Servlet API. Solo hay que agregar JUnit 5 y H2 como dependencias de prueba:

```
<!-- Agregar en la sección <dependencies> del pom.xml existente -->

<!-- JUnit 5 - framework de pruebas (scope test: no va en el WAR) -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>

<!-- H2 - BD en memoria para pruebas de integración (scope test) -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.2.224</version>
  <scope>test</scope>
</dependency>

<!-- Agregar en la sección <build><plugins> -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
</plugin>
```

3.2 Estructura de carpetas de prueba

```
captcha/src/
├── main/java/sena/adso/captcha/
│   ├── dto/
│   │   ├── Paciente.java           ← ya existe
│   │   ├── Vacuna.java             ← ya existe
│   │   ├── Usuario.java            ← ya existe
│   │   └── RegistroVacunacion.java ← ya existe
│   ├── dao/
│   │   ├── PacienteDAO.java         ← ya existe
│   │   ├── VacunaDAO.java           ← ya existe
│   │   └── RegistroVacunacionDAO.java ← ya existe
│   ├── model/
│   │   └── Conexion.java            ← ya existe
│   └── util/
│       └── ValidadorDocumento.java ← NUEVA clase a crear para probar
```

```

└─ test/java/sena/adso/captcha/
    └─ dto/
        ├── PacienteTest.java           ← Sesión 1
        ├── VacunaTest.java             ← Sesión 1
        └── UsuarioTest.java            ← Sesión 1
    └─ util/
        └── ValidadorDocumentoTest.java ← Sesión 2
    └─ dao/
        ├── ConexionTestHelper.java     ← clase auxiliar para H2
        ├── PacienteDAOTest.java         ← Sesión 3
        ├── VacunaDAOTest.java           ← Sesión 3
        └── RegistroVacunacionDAOTest.java ← Sesión 3

captcha/src/test/resources/
└─ db-test.properties                 ← configuración de H2

```

4. Nueva Clase: ValidadorDocumento.java

Para practicar `@ParameterizedTest` necesitamos una clase con lógica de validación. Crea esta clase en `src/main/java/sena/adso/captcha/util/`:

```

package sena.adso.captcha.util;

/**
 * Valida números de documento colombianos.
 * Reglas: solo dígitos, entre 6 y 10 caracteres, no null ni vacío.
 */
public class ValidadorDocumento {

    private static final int MIN_DIGITOS = 6;
    private static final int MAX_DIGITOS = 10;

    /**
     * Valida si un número de documento es válido.
     * @param documento Número de documento a validar
     * @return true si es válido, false si no
     */
    public boolean esValido(String documento) {
        if (documento == null || documento.isBlank()) return false;
        String limpio = documento.trim();
        if (!limpio.matches("\\d+")) return false;
        return limpio.length() >= MIN_DIGITOS && limpio.length() <= MAX_DIGITOS;
    }

    /**
     * Verifica si el documento pertenece a un menor de edad
     * (tarjeta de identidad: 10 dígitos empezando por 1).
     */
    public boolean esTarjetaIdentidad(String documento) {
        if (!esValido(documento)) return false;
        return documento.trim().length() == 10
            && documento.trim().startsWith("1");
    }
}

```

También necesitamos agregar el método `estaVencida()` a `Vacuna.java` y los métodos `esMedico()/esEnfermero()` a `Usuario.java` para tener más lógica que probar:

5. Sesión 1 — Pruebas Unitarias de los DTOs

Objetivo de la sesión 1

Escribir pruebas unitarias para Paciente.java, Vacuna.java y Usuario.java.

Aprender @BeforeEach, assertAll, assertThrows y @Nested.

NO se usa base de datos en esta sesión — todo en memoria.

5.1 PacienteTest.java

```

    }

    // — getNombreCompleto() —————

    @Test
    @DisplayName("getNombreCompleto() debe unir nombres y apellidos con espacio")
    void getNombreCompleto_debeUnirNombresYApellidos() {
        // ARRANGE — ya listo en @BeforeEach
        // ACT
        String resultado = paciente.getNombreCompleto();
        // ASSERT
        assertEquals("Carlos Ernesto Pedraza Rondón", resultado);
    }

    @Test
    @DisplayName("getNombreCompleto() con nombres vacíos debe retornar solo apellidos")
    void getNombreCompleto_conNombresVacios() {
        paciente.setNombres("");
        // Nota: depende de cómo esté implementado getNombreCompleto()
        // Esta prueba documenta el comportamiento actual del sistema
        String resultado = paciente.getNombreCompleto();
        assertTrue(resultado.contains("Pedraza Rondón"));
    }

    // — assertAll: verificar todos los campos a la vez —————

    @Test
    @DisplayName("Constructor debe inicializar todos los campos correctamente")
    void constructor_debeInicializarTodosLosCampos() {
        assertAll("Campos del Paciente",
            () -> assertEquals(1, paciente.getId()),
            () -> assertEquals("Carlos Ernesto", paciente.getNombres()),
            () -> assertEquals("Pedraza Rondón", paciente.getApellidos()),
            () -> assertEquals("1052345678", paciente.getDocumento()),
            () -> assertNotNull(paciente.getFechaNacimiento())
        );
    }

    // — Getters y Setters —————

    @Test
    @DisplayName("setDocumento() debe actualizar el documento correctamente")
    void setDocumento_debeActualizarDocumento() {
        paciente.setDocumento("80456789");
        assertEquals("80456789", paciente.getDocumento());
    }

    @Test
    @DisplayName("Paciente vacío (constructor sin parámetros) debe tener id=0")
    void constructorVacio_debeCrearPacienteConIdCero() {
        Paciente vacio = new Paciente();
        assertEquals(0, vacio.getId());
        assertNull(vacio.getNombres());
    }

    // — @Nested: agrupar por contexto —————

    @Nested

```

```

@DisplayName("Pruebas de nombre completo")
class NombreCompletoTests {

    @Test
    @DisplayName("Nombre completo de paciente real de Nobsa")
    void pacienteNobsa_nombreCompleto() {
        Paciente nobsa = new Paciente(2, "María Eugenia", "Suárez Cely",
            "1052345679", new Date());

        assertEquals("María Eugenia Suárez Cely", nobsa.getNombreCompleto());
    }

    @Test
    @DisplayName("Nombre completo no debe ser null")
    void nombreCompleto_noDebeSerNull() {
        assertNotNull(paciente.getNombreCompleto());
    }

    @Test
    @DisplayName("Nombre completo no debe estar vacío")
    void nombreCompleto_noDebeEstarVacio() {
        assertFalse(paciente.getNombreCompleto().isBlank());
    }
}

```

5.2 VacunaTest.java

```

package sena.adso.captcha.dto;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class VacunaTest {

    private Vacuna vacunaVigente;
    private Vacuna vacunaVencida;

    @BeforeEach
    void setUp() {
        // Vacuna con fecha futura (datos del script vacunacion_db.sql)
        vacunaVigente = new Vacuna(1, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-31");
        // Vacuna con fecha pasada (vencida)
        vacunaVencida = new Vacuna(3, "Influenza", "IN001", "Sanofi", "2024-01-01");
    }

    @Test
    @DisplayName("getInfoCompleta() debe retornar formato: Nombre (Lote: X, Lab: Y)")
    void getInfoCompleta_debeRetornarFormatoCorrecto() {
        String esperado = "COVID-19 Pfizer (Lote: PF001, Lab: Pfizer)";
        assertEquals(esperado, vacunaVigente.getInfoCompleta());
    }

    @Test
    @DisplayName("estaVencida() debe retornar false para vacuna con fecha futura")

```



```

void estaVencida_conFechaFutura_debeRetornarFalse() {
    assertFalse(vacunaVigente.estaVencida());
}

@Test
@DisplayName("estaVencida() debe retornar true para vacuna con fecha pasada")
void estaVencida_conFechaPasada_debeRetornarTrue() {
    assertTrue(vacunaVencida.estaVencida());
}

@Test
@DisplayName("Constructor debe asignar todos los campos correctamente")
void constructor_debeAsignarTodosLosCampos() {
    assertEquals("Campos de Vacuna",
        () -> assertEquals(1, vacunaVigente.getId()),
        () -> assertEquals("COVID-19 Pfizer", vacunaVigente.getNombre()),
        () -> assertEquals("PF001", vacunaVigente.getLote()),
        () -> assertEquals("Pfizer", vacunaVigente.getLaboratorio()),
        () -> assertEquals("2026-12-31",
            vacunaVigente.getFechaVencimiento())
    );
}

@Test
@DisplayName("Vacuna con fecha_vencimiento null no debe lanzar excepción")
void estaVencida_conFechaNull_debeRetornarFalse() {
    Vacuna sinFecha = new Vacuna(5, "Triple Viral", "TV001", "Serum", null);
    // No debe lanzar NullPointerException - el método debe manejarlo
    assertDoesNotThrow(() -> sinFecha.estaVencida());
    assertFalse(sinFecha.estaVencida());
}
}

```

5.3 UsuarioTest.java

```

package sena.adso.captcha.dto;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class UsuarioTest {

    private Usuario medico;
    private Usuario enfermero;

    @BeforeEach
    void setUp() {
        // Datos del INSERT de muestra en vacunacion_db.sql
        medico = new Usuario(1, "Admin", "Sistema", "10000000000",
            "admin@example.com", "admin", "admin123",
            "MEDICO", "Administración", "Hospital General");
        enfermero = new Usuario(2, "María", "López", "52345678",
            "maria@hospital.com", "mlopez", "enf123",
            "ENFERMERO", null, "Hospital General");
    }

    @Test

```

```

    @DisplayName("esMedico() debe retornar true para usuario con rol MEDICO")
    void esMedico_conRolMedico_debeRetornarTrue() {
        assertTrue(medico.esMedico());
    }

    @Test
    @DisplayName("esMedico() debe retornar false para usuario con rol ENFERMERO")
    void esMedico_conRolEnfermero_debeRetornarFalse() {
        assertFalse(enfermero.esMedico());
    }

    @Test
    @DisplayName("esEnfermero() debe retornar true para usuario con rol ENFERMERO")
    void esEnfermero_conRolEnfermero_debeRetornarTrue() {
        assertTrue(enfermero.esEnfermero());
    }

    @Test
    @DisplayName("esMedico() debe ser insensible a mayúsculas (medico == MEDICO)")
    void esMedico_debeSerInsensibleAMayusculas() {
        medico.setRol("medico"); // minúsculas
        assertTrue(medico.esMedico());
    }

    @Test
    @DisplayName("getNombreCompleto() para el admin del sistema")
    void getNombreCompleto_paraAdmin() {
        assertEquals("Admin Sistema", medico.getNombreCompleto());
    }

    @Test
    @DisplayName("Usuario con rol null NO debe ser médico ni enfermero")
    void usuarioSinRol_noEsMedicoNiEnfermero() {
        medico.setRol(null);
        assertFalse(medico.esMedico());
        assertFalse(medico.esEnfermero());
    }
}

```

6. Sesión 2 — Pruebas Parametrizadas con @ParameterizedTest

Objetivo de la sesión 2

Usar @ParameterizedTest para probar ValidadorDocumento.java con muchos casos sin duplicar código. Entender los casos límite del sistema de vacunación.

6.1 ValidadorDocumentoTest.java

```

package sena.adso.captcha.util;

import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

```

```

class ValidadorDocumentoTest {

    private ValidadorDocumento validador;

    @BeforeEach
    void setUp() { validador = new ValidadorDocumento(); }

    // — Documentos válidos — datos reales del script vacunacion_db.sql —

    @ParameterizedTest(name = "Documento válido: [{0}]")
    @ValueSource(strings = {
        "1000000000", // admin del sistema (10 dígitos)
        "1052345678", // paciente de Nobsa
        "52345678",   // 8 dígitos — válido
        "123456",     // 6 dígitos — mínimo válido
        "9999999999"  // 10 dígitos — máximo válido
    })
    @DisplayName("Documentos válidos deben retornar true")
    void documentoValido_debeRetornarTrue(String doc) {
        assertTrue(validador.esValido(doc),
            "Se esperaba que '" + doc + "' fuera válido");
    }

    // — Documentos inválidos —————

    @ParameterizedTest(name = "Documento inválido: [{0}]")
    @ValueSource(strings = {
        "12345",      // 5 dígitos — menor al mínimo
        "12345678901", // 11 dígitos — mayor al máximo
        "CC-123456",   // tiene letras y guión
        "105 345",     // tiene espacio
        ""            // vacío
    })
    @DisplayName("Documentos inválidos deben retornar false")
    void documentoInvalido_debeRetornarFalse(String doc) {
        assertFalse(validador.esValido(doc));
    }

    @Test
    @DisplayName("null debe retornar false sin lanzar NullPointerException")
    void documentoNull_debeRetornarFalse() {
        assertFalse(validador.esValido(null));
    }

    // — @CsvSource: probar esTarjetaIdentidad() con input + expected —

    @ParameterizedTest(name = "doc={0}, esperado={1}")
    @CsvSource({
        "1052345678, true", // 10 dígitos, empieza en 1 → tarjeta
        "1000000000, true", // admin del sistema → tarjeta
        "52345678, false",  // 8 dígitos → no es tarjeta
        "2052345678, false", // 10 dígitos pero empieza en 2 → no es tarjeta
        "123456, false",    // 6 dígitos → no es tarjeta
    })
    @DisplayName("esTarjetaIdentidad() con varios documentos")
    void esTarjetaIdentidad_variosDocumentos(String doc, boolean esperado) {
        assertEquals(esperado, validador.esTarjetaIdentidad(doc.trim()));
    }
}

```

```
// — Pruebas de casos límite —————

@Test
@DisplayName("Documento con exactamente 6 dígitos (mínimo) debe ser válido")
void documento_6digitos_debeSerValido() {
    assertTrue(validador.esValido("123456")); // valor límite mínimo
}

@Test
@DisplayName("Documento con exactamente 10 dígitos (máximo) debe ser válido")
void documento_10digitos_debeSerValido() {
    assertTrue(validador.esValido("1052345678")); // valor límite máximo
}

@Test
@DisplayName("Documento con 5 dígitos (debajo del mínimo) debe ser inválido")
void documento_5digitos_debeSerInvalido() {
    assertFalse(validador.esValido("12345")); // un dígito por debajo
}

@Test
@DisplayName("Documento con 11 dígitos (encima del máximo) debe ser inválido")
void documento_11digitos_debeSerInvalido() {
    assertFalse(validador.esValido("10523456780")); // un dígito por encima
}
}
```

6.2 Tabla de casos límite del sistema de vacunación

Regla del sistema	Valor límite	Caso de prueba	¿Qué puede fallar?
Documento mínimo 6 dígitos	123456 (exacto)	assertTrue(esValido("123456"))	Rechazar el valor mínimo válido
Documento máximo 10 dígitos	1052345678 (exacto)	assertTrue(esValido("1052345678"))	Rechazar el valor máximo válido
Vacuna: no duplicar lote+nombre	Mismo lote PF001 dos veces	assertThrows en VacunaDAO.insertar()	No detectar la violación UNIQUE
Vacuna no vencida	Fecha vencimiento = hoy	assertFalse(estaVencida())	Marcar como vencida el mismo día
getNombreCompleto no null	nombres = null	assertNotNull(getNombreCompleto())	NullPointerException al concatenar
Rol insensible a mayúsculas	"medico" vs "MEDICO"	assertTrue(esMedico()) con minúsculas	Rechazar rol en minúsculas

7. Sesión 3 — Pruebas de Integración con H2

¿Por qué H2 y no MySQL directamente?

Las pruebas de integración necesitan una base de datos REAL, pero usar MySQL de producción tiene riesgos: datos sucios entre pruebas, conexión compartida con otros, velocidad lenta. H2 es la solución:

- BD en memoria: se crea al iniciar las pruebas y se borra al terminar.
- Cada prueba empieza con datos limpios — garantiza independencia.
- Soporta MODE=MySQL: el mismo SQL del sistema de vacunación funciona en H2.
- 10x más rápida que MySQL para pruebas.
- scope test: H2 nunca va al WAR de producción.

7.1 db-test.properties — Configuración de H2

```
# src/test/resources/db-test.properties
# Configuración de H2 en memoria para pruebas de integración

db.url=jdbc:h2:mem:vacunacion_test;DB_CLOSE_DELAY=-1;MODE=MySQL
db.driver=org.h2.Driver
db.user=sa
db.password=

# DB_CLOSE_DELAY=-1 → la BD permanece en memoria mientras la JVM corre
# MODE=MySQL → H2 acepta sintaxis MySQL (AUTO_INCREMENT, ENUM, etc.)
```

7.2 ConexionTestHelper.java — Clase auxiliar para pruebas

```
package sena.adso.captcha.dao;

import java.io.InputStream;
import java.sql.*;
import java.util.Properties;

/**
 * Clase auxiliar SOLO para pruebas.
 * Provee una conexión H2 en lugar de MySQL.
 */
public class ConexionTestHelper {

    private static Connection conexion;

    public static Connection getConexion() throws Exception {
        if (conexion == null || conexion.isClosed()) {
            Properties props = new Properties();

```

```

        try (InputStream in = ConexionTestHelper.class
                .getClassLoader()
                .getResourceAsStream("db-test.properties")) {
            props.load(in);
        }
        Class.forName(props.getProperty("db.driver"));
        conexion = DriverManager.getConnection(
            props.getProperty("db.url"),
            props.getProperty("db.user"),
            props.getProperty("db.password")
        );
    }
    return conexion;
}

public static void ejecutarSQL(String sql) throws Exception {
    try (Statement stmt = getConexion().createStatement()) {
        stmt.execute(sql);
    }
}
}

```

7.3 PacienteDAOTest.java — Integración completa

```

package sena.adso.captcha.dao;

import sena.adso.captcha.dto.Paciente;
import org.junit.jupiter.api.*;
import java.util.*;
import static org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de INTEGRACIÓN: PacienteDAO contra H2 en memoria.
 * El mismo SQL del script vacunacion_db.sql funciona en H2 con MODE=MySQL.
 */
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class PacienteDAOTest {

    private static PacienteDAO dao;

    @BeforeAll
    static void crearTabla() throws Exception {
        // Crear la tabla pacientes en H2 - mismo DDL del script vacunacion_db.sql
        ConexionTestHelper.ejecutarSQL(
            "CREATE TABLE IF NOT EXISTS pacientes (" +
            "    id INT AUTO_INCREMENT PRIMARY KEY," +
            "    nombres VARCHAR(100) NOT NULL," +
            "    apellidos VARCHAR(100) NOT NULL," +
            "    documento VARCHAR(20) NOT NULL UNIQUE," +
            "    fecha_nacimiento DATE NOT NULL," +
            "    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)"
        );
        dao = new PacienteDAO(); // usar constructor sin parámetros
        // NOTA: PacienteDAO necesita ser adaptado para aceptar una Connection
        // externa, o usar una variable de entorno para la URL de BD.
        // Ver sección 7.4 para la adaptación mínima requerida.
    }
}

```

```

@BeforeEach
void limpiarDatos() throws Exception {
    // Limpiar antes de CADA prueba – independencia total
    ConexionTestHelper.ejecutarSQL("DELETE FROM pacientes");
}

@AfterAll
static void eliminarTabla() throws Exception {
    ConexionTestHelper.ejecutarSQL("DROP TABLE IF EXISTS pacientes");
}

@Test
@Order(1)
@DisplayName("insertar() debe retornar true y guardar el paciente en H2")
void insertar_debeRetornarTrueYGuardarPaciente() throws Exception {
    Paciente p = new Paciente(0, "Carlos Ernesto", "Pedraza Rondón",
        "1052345678",
        java.sql.Date.valueOf("1985-03-12"));

    boolean resultado = dao.insertar(p);

    assertTrue(resultado, "insertar() debe retornar true al guardar el
paciente");
}

@Test
@Order(2)
@DisplayName("listarTodos() debe retornar lista vacía si no hay pacientes")
void listarTodos_debeRetornarListaVacía_siBDVacía() throws Exception {
    List<Paciente> lista = dao.listarTodos();
    assertNotNull(lista, "La lista nunca debe ser null");
    assertTrue(lista.isEmpty(), "Sin datos, la lista debe estar vacía");
}

@Test
@Order(3)
@DisplayName("listarTodos() debe retornar los pacientes insertados")
void listarTodos_debeRetornarPacientesInsertados() throws Exception {
    // Insertar dos pacientes de Boyacá
    dao.insertar(new Paciente(0, "Carlos Ernesto", "Pedraza Rondón",
        "1052345678",
        java.sql.Date.valueOf("1985-03-12")));
    dao.insertar(new Paciente(0, "María Eugenia", "Suárez Cely",
        "1052345679",
        java.sql.Date.valueOf("1992-07-24")));

    List<Paciente> lista = dao.listarTodos();

    assertEquals(2, lista.size(), "Deben existir exactamente 2 pacientes");
}

@Test
@Order(4)
@DisplayName("insertar() con documento duplicado debe fallar (UNIQUE
constraint)")
void insertar_conDocumentoDuplicado_debeFallar() throws Exception {
    Paciente p1 = new Paciente(0, "Carlos", "Pedraza",
        "1052345678",

```

```

        java.sql.Date.valueOf("1985-03-12"));
    Paciente p2 = new Paciente(0, "Otro", "Nombre",
        "1052345678", // mismo documento
        java.sql.Date.valueOf("1990-01-01"));

    dao.insertar(p1); // primer insert exitoso
    boolean resultado = dao.insertar(p2); // debe fallar

    assertFalse(resultado,
        "No se debe poder insertar dos pacientes con el mismo documento");
}

@Test
@Order(5)
@DisplayName("eliminar() debe reducir el conteo de pacientes en 1")
void eliminar_debeReducirConteo() throws Exception {
    dao.insertar(new Paciente(0, "Carlos", "Pedraza",
        "1052345678",
        java.sql.Date.valueOf("1985-03-12")));

    List<Paciente> antes = dao.listarTodos();
    int idEliminar = antes.get(0).getId();

    dao.eliminar(idEliminar);

    List<Paciente> despues = dao.listarTodos();
    assertEquals(antes.size() - 1, despues.size());
}
}

```

7.4 Adaptación mínima de PacienteDAO para pruebas

El PacienteDAO actual usa `Conexion.getConnection()` que apunta a MySQL. Para las pruebas de integración necesita poder usar la conexión H2. La forma más sencilla sin refactorizar el DAO completo es usar variables de entorno:

```

// Opción A – Agregar constructor que acepta Connection (recomendada):
public class PacienteDAO {

    private Connection conexionExterna; // null = usar Conexion.getConnection()

    public PacienteDAO() { this.conexionExterna = null; }

    // Constructor para pruebas de integración
    public PacienteDAO(Connection conexion) {
        this.conexionExterna = conexion;
    }

    private Connection obtenerConexion() throws SQLException {
        return conexionExterna != null
            ? conexionExterna
            : Conexion.getConnection();
    }

    // ... resto del DAO usa obtenerConexion() en lugar de
    Conexion.getConnection()
}

```



```
// En la prueba de integración:
dao = new PacienteDAO(ConexionTestHelper.getConexion()); // usa H2

// En producción (Servlet):
PacienteDAO dao = new PacienteDAO(); // usa MySQL - sin cambios
```

7.5 VacunaDAOTest.java — Prueba de la restricción UNIQUE

```
package sena.adso.captcha.dao;

import sena.adso.captcha.dto.Vacuna;
import org.junit.jupiter.api.*;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class VacunaDAOTest {

    private static VacunaDAO dao;

    @BeforeAll
    static void crearTabla() throws Exception {
        // La tabla vacunas tiene UNIQUE KEY uk_lote_nombre (lote, nombre)
        ConexionTestHelper.ejecutarSQL(
            "CREATE TABLE IF NOT EXISTS vacunas (" +
            "  id INT AUTO_INCREMENT PRIMARY KEY," +
            "  nombre VARCHAR(100) NOT NULL," +
            "  lote VARCHAR(50) NOT NULL," +
            "  laboratorio VARCHAR(100) NOT NULL," +
            "  fecha_vencimiento DATE NOT NULL," +
            "  UNIQUE KEY uk_lote_nombre (lote, nombre))"
        );
        dao = new VacunaDAO(ConexionTestHelper.getConexion());
    }

    @BeforeEach
    void limpiar() throws Exception {
        ConexionTestHelper.ejecutarSQL("DELETE FROM vacunas");
    }

    @AfterAll
    static void destruir() throws Exception {
        ConexionTestHelper.ejecutarSQL("DROP TABLE IF EXISTS vacunas");
    }

    @Test
    @Order(1)
    @DisplayName("insertar() debe guardar la vacuna en H2")
    void insertar_debeGuardarVacuna() {
        // Datos del INSERT del script vacunacion_db.sql
        Vacuna v = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-31");
        assertTrue(dao.insertar(v));
    }

    @Test
    @Order(2)
```

```

    @DisplayName("insertar() mismo lote+nombre debe fallar (UNIQUE KEY)")
    void insertar_loteDuplicado_debeFallar() {
        Vacuna v1 = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-31");
        Vacuna v2 = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2027-01-01");

        dao.insertar(v1);
        boolean resultado = dao.insertar(v2); // mismo lote+nombre
        assertFalse(resultado,
            "No se puede insertar dos vacunas con el mismo lote y nombre");
    }

    @Test
    @Order(3)
    @DisplayName("listarTodos() retorna las vacunas del script de datos de prueba")
    void listarTodos_retornaVacunasInsertadas() {
        // Insertar las 5 vacunas del script vacunacion_db.sql
        dao.insertar(new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-31"));
        dao.insertar(new Vacuna(0, "COVID-19 Moderna", "MD001", "Moderna", "2026-10-15"));
        dao.insertar(new Vacuna(0, "Influenza", "IN001", "Sanofi", "2025-08-22"));
        dao.insertar(new Vacuna(0, "Hepatitis B", "HB001", "GSK", "2026-05-10"));
        dao.insertar(new Vacuna(0, "Triple Viral (SRP)", "TV001", "Serum Institute", "2025-11-30"));

        List<Vacuna> lista = dao.listarTodos();
        assertEquals(5, lista.size());
    }
}

```

8. Ejecutar las Pruebas

8.1 Desde la terminal

```

# Desde la raíz del proyecto captcha/

# Ejecutar TODAS las pruebas
mvn test

# Ejecutar solo PacienteTest (unitarias)
mvn test -Dtest=PacienteTest

# Ejecutar solo las pruebas de integración (DAO)
mvn test -Dtest="*DAOTest"

# Ejecutar solo las pruebas unitarias (excluir DAO)
mvn test -Dexcludes="**/dao/*Test.java"

# Ver reporte detallado en consola
mvn test -Dsurefire.reportFormat=plain

# Resultado exitoso esperado:
[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

```

8.2 Interpretar el reporte

```
# Prueba exitosa:
[INFO] PacienteTest > getNombreCompleto_debeUnirNombresYApellidos PASSED

# Prueba fallida — cómo leerla:
[ERROR] VacunaTest > estaVencida_conFechaFutura_debeRetornarFalse FAILED
org.opentest4j.AssertionFailedError:
    expected: <false> but was: <true>
    at VacunaTest.estaVencida_conFechaFutura_debeRetornarFalse(VacunaTest.java:31)

# Lectura: esperaba false (no vencida) pero obtuvo true (vencida)
# → la lógica de estaVencida() tiene un error

# Error de integración — BD no accesible:
[ERROR] PacienteDAOTest > insertar_debeRetornarTrue FAILED
java.lang.NullPointerException: db-test.properties no encontrado
# → verificar que el archivo está en src/test/resources/
```

9. Ejercicios Prácticos por Sesión

Distribución — 3 sesiones de laboratorio

Sesión 1 (4 h): Pruebas unitarias de DTOs — Paciente, Vacuna y Usuario.

Sesión 2 (4 h): @ParameterizedTest con ValidadorDocumento + casos límite.

Sesión 3 (4 h): Pruebas de integración con H2 — PacienteDAO y VacunaDAO.

9.1 Sesión 1 — DTOs del Sistema de Vacunación

#	Actividad
1	Agrega las dependencias de JUnit 5 y H2 al pom.xml del sistema de vacunación. Ejecuta mvn test — debe mostrar 'No tests found' pero sin errores de compilación.
2	Agrega esMedico() y esEnfermero() a Usuario.java y estaVencida() a Vacuna.java (sección 4). Compila con mvn compile.
3	Crea PacienteTest.java con las 6 pruebas de la sección 5.1. Ejecuta y verifica que todas pasan con barra verde.
4	Crea VacunaTest.java con las 5 pruebas de la sección 5.2. Presta especial atención a la prueba de vacuna con fecha null.
5	Crea UsuarioTest.java con las 6 pruebas de la sección 5.3. Verifica que la prueba de rol en minúsculas pasa — si falla, ajusta el método esMedico().
6	RETO: Agrega a RegistroVacunacion.java un método tieneObservaciones() que retorna true si observaciones no es null ni vacío. Escribe 3 pruebas.
7	ANÁLISIS: Ejecuta las pruebas de PacienteTest y mide el tiempo (mvn test). ¿Cuántos milisegundos tardan? ¿Por qué son tan rápidas si no usan BD?
8	REFLEXIÓN: Escribe intencionalmente una prueba que falle cambiando el valor esperado en assertEquals. Lee el mensaje de error. ¿Qué información te da para corregir el bug?

9.2 Sesión 2 — Pruebas Parametrizadas

#	Actividad
1	Crea ValidadorDocumento.java en src/main/java/sena/adso/captcha/util/ (sección 4). Compila con mvn compile.
2	Crea ValidadorDocumentoTest.java con los dos @ParameterizedTest de @ValueSource (documentos válidos e inválidos). Ejecuta — deben aparecer 10 casos individuales en el reporte.
3	Agrega la prueba de @CsvSource para esTarjetaIdentidad() con los 5 escenarios. Verifica que todos pasan.
4	Escribe los 4 casos límite explícitos (6 dígitos mínimo, 10 máximo, 5 dígitos inválido, 11 dígitos inválido). Explica en tu bitácora por qué los bordes son los más propensos a errores.
5	RETO: Agrega a ValidadorDocumento un método esFormatoCedula(String doc) que acepta formatos como '1.052.345.678' (con puntos). Escribe pruebas con @ParameterizedTest.
6	Completa la tabla de casos límite (sección 6.2) con 2 casos propios del sistema de vacunación que aún no están cubiertos.
7	COMPARACIÓN: ¿Cuántas líneas de código tendrías si no usaras @ParameterizedTest y escribieras las 10 pruebas de documentos como @Test individuales? Calcula y anota.

9.3 Sesión 3 — Pruebas de Integración

#	Actividad
1	Crea db-test.properties en src/test/resources/ y ConexionTestHelper.java (sección 7.2). Escribe una prueba mínima que solo verifica la conexión H2.
2	Adapta PacienteDAO.java para aceptar una Connection externa por constructor (sección 7.4). Verifica que el sistema sigue funcionando en Tomcat.
3	Crea PacienteDAOTest.java con las 5 pruebas de la sección 7.3. Ejecuta y verifica que todas pasan. Observa el @BeforeEach limpiando los datos.
4	Crea VacunaDAOTest.java con las 3 pruebas de la sección 7.5. La prueba de UNIQUE KEY es la más importante — verifica que el DAO maneja correctamente el error SQL.
5	RETO: Crea RegistroVacunacionDAOTest con una prueba de integración que: inserta un paciente, inserta una vacuna, inserta un registro de vacunación y verifica que listarTodos() retorna 1 registro.
6	COMPARACIÓN: Ejecuta mvn test -Dtest='*DAOTest' y anota el tiempo. Luego ejecuta solo los tests unitarios y anota el tiempo. ¿Cuántas veces más lentos son los de integración?
7	RETO AVANZADO: Agrega a PacienteDAOTest una prueba con assertTimeout() que verifique que listarTodos() con 50 pacientes responde en menos de 500ms en H2.

10. Preguntas de Reflexión

 Responde en tu bitácora — mínimo 5 líneas por pregunta

Las respuestas se socializan en plenaria al cierre de la sesión 3.

10.1 Sobre Pruebas Unitarias

- En VacunaTest.java usamos @BeforeEach para crear vacunaVigente y vacunaVencida antes de cada prueba. ¿Qué pasaría si modificaras vacunaVigente en una prueba y la siguiente prueba dependiera de ese estado? ¿Cómo garantiza @BeforeEach que esto no ocurra?
- La prueba estaVencida_conFechaNull_debeRetornarFalse() usa assertDoesNotThrow(). ¿Por qué es importante probar que el método no lanza una excepción inesperada? En el sistema de vacunación, ¿dónde podría ocurrir un NullPointerException no controlado si no se prueban estos casos?
- assertAll() en el constructor de Paciente ejecuta todos los assertions aunque alguno falle. Si usaras assertEquals() individual y el primer campo falla, ¿cuántos campos no verías en el reporte? ¿Cuándo usarías assertAll() en el sistema de vacunación?

10.2 Sobre Pruebas Parametrizadas

- @ParameterizedTest con 10 documentos genera 10 casos de prueba desde un solo método. Imagina que el registrador de Nobsa te informa que ahora deben aceptar documentos de 11 dígitos para extranjeros. ¿Cuántas líneas de código cambiarías en ValidadorDocumentoTest? ¿Y si hubieran sido 10 métodos @Test separados?
- Los casos límite (6 dígitos mínimo, 10 máximo, etc.) son donde más errores se esconden. En el sistema de vacunación, ¿cuál es el caso límite más crítico en términos de impacto para el paciente? (Pista: piensa en fechas de vencimiento de vacunas).

10.3 Sobre Pruebas de Integración

- La prueba insertar_conDocumentoDuplicado_debeFallar() verifica que el DAO retorna false cuando viola la restricción UNIQUE. En el sistema de vacunación en producción, ¿qué mostraría el usuario si este error no estuviera manejado? ¿Cómo mejorarías el mensaje de error al usuario?
- Usamos H2 en lugar de MySQL para las pruebas de integración. Tu compañero propone conectar las pruebas directamente a la BD MySQL del sistema de vacunación en producción. Menciona exactamente 3 problemas que podrían ocurrir en la clase de hoy si hiciera eso.
- @BeforeEach ejecuta DELETE FROM pacientes antes de cada prueba. ¿Por qué es crítico este paso en PacienteDAOTest? Escribe el escenario exacto donde la prueba listarTodos_debeRetornarPacientesInsertados() fallaría si no limpiaras antes.

10.4 Prueba Final

 Aplicación de Citas Médicas

Aplique todo lo aprendido en la aplicación construida.

11. Errores Frecuentes y Soluciones

Error / Síntoma	Causa y Solución
No tests were found en el proyecto de vacunación	Maven no detecta las pruebas. El archivo de prueba debe terminar en Test.java y estar en src/test/java/ (no en src/main/).
estaVencida() NullPointerException con fecha null	LocalDate.parse(null) lanza excepción. Agregar: if (fechaVencimiento == null fechaVencimiento.isBlank()) return false; al inicio del método.
db-test.properties not found (H2 no conecta)	El archivo está en la ruta incorrecta. Debe estar en src/test/resources/ (no en src/main/resources/). Verifica con: ls src/test/resources/
Table 'pacientes' not found en la prueba de integración	El @BeforeAll que crea la tabla no se ejecutó. Verifica que sea static void y que la anotación sea de org.junit.jupiter.api.
La prueba de UNIQUE KEY pasa aunque inserte el duplicado	El PacienteDAO captura la excepción y retorna false — ese es el comportamiento correcto. Si retorna true con duplicado, hay un bug real en el DAO.
Pruebas pasan solas pero fallan en conjunto	Dependencia de estado entre pruebas: una prueba modifica datos que otra necesita limpios. Usar @BeforeEach para resetear.
esMedico() retorna false con rol en minúsculas	La comparación usa == en lugar de equalsIgnoreCase(). Cambiar: return "MEDICO".equalsIgnoreCase(rol);
@ParameterizedTest no ejecuta todos los casos	Falta el import: import org.junit.jupiter.params.ParameterizedTest; y import org.junit.jupiter.params.provider.ValueSource;

12. Criterios de Evaluación

#	Criterio de Desempeño	Puntos	Instrumento
1	PacienteTest.java: mínimo 5 pruebas incluyendo assertAll y @Nested — todas pasan	20 %	mvn test
2	VacunaTest.java: prueba de estaVencida() con fecha futura, pasada y null — todas pasan	15 %	mvn test
3	UsuarioTest.java: esMedico() y esEnfermero() incluyendo prueba con rol en minúsculas	10 %	mvn test
4	ValidadorDocumentoTest.java: @ParameterizedTest con mínimo 5 documentos válidos y 5 inválidos	15 %	Revisión código
5	PacienteDAOTest.java: 5 pruebas de integración con H2 incluyendo UNIQUE — todas pasan	20 %	mvn test
6	VacunaDAOTest.java: 3 pruebas incluyendo la de lote duplicado — todas pasan	10 %	mvn test
7	Preguntas de reflexión respondidas (sección 10)	10 %	Lista de chequeo

#	Criterio de Desempeño	Puntos	Instrumento
BONUS A	RegistroVacunacionDAOTest: flujo completo paciente+vacuna+registro	+0,3	mvn test
BONUS B	assertTimeout() verificando rendimiento de listarTodos() con 50 registros	+0,2	Demostración

Condiciones de no aprobación automática

- Pruebas que siempre pasan sin importar el código (assertions vacíos o triviales).
- Pruebas de integración que apuntan a MySQL de producción (no a H2).
- Pruebas con nombres genéricos test1(), test2() sin @DisplayName descriptivo.
- Pruebas que dependen del orden de ejecución sin @TestMethodOrder.
- Menos de 5 pruebas en cualquier clase marcada como obligatoria.

SENA · Servicio Nacional de Aprendizaje

Centro Industrial de Mantenimiento y Manufactura – CIMM | Regional Boyacá
Tecnólogo en Análisis y Desarrollo de Software – ADSO | 2026